PROTOTIPO MÍNIMO Y DECISIONES TÉCNICAS

1. Justificación de Decisiones Técnicas 2

2. Criterios de Éxito del Proyecto 3

3. Información para ejecutar el proyecto 4

4. Verificación del prototipo 5

Carlos Flores Hernández 2° DAW

1. Justificación de Decisiones Técnicas

En esta fase de prototipo, he tomado las siguientes decisiones para garantizar un sistema robusto y escalable:

Arquitectura Desacoplada (Decoupled): He separado totalmente el Frontend (React) del Backend (Node.js/Express). Esto permite que la interfaz sea rápida e independiente de la lógica del servidor, facilitando el mantenimiento y posibles ampliaciones futuras.

Uso de Sequelize (ORM): Para la comunicación con la base de datos, utilizo Sequelize. Esto me permite modelar los datos como objetos de JavaScript, mejorando la legibilidad del código y proporcionando una capa de seguridad automática contra ataques de SQL Injection.

Conexión mediante Session Pooler (Puerto 6543): He configurado la cadena de conexión a través del puerto 6543 de Supabase. Esta decisión técnica es crítica para asegurar la compatibilidad con redes IPv4 (evitando fallos de conexión en redes que no soportan IPv6) y garantizar la estabilidad de las sesiones persistentes que requiere el servidor.

Gestión de Estados en el Frontend: Utilizo React Hooks (useState, useEffect) para manejar el tema (oscuro/claro), los roles de usuario y la validación de formularios en tiempo real, lo que ofrece una experiencia de usuario fluida sin recargas de página.

Seguridad de Credenciales: El uso de variables de entorno (.env) y su exclusión del repositorio mediante .gitignore asegura que las claves de acceso a la base de datos nunca queden expuestas públicamente.

2. Criterios de Éxito del Proyecto

Para validar el proyecto final como exitoso, comprobaré los siguientes puntos:

Autenticación y Redirección por Rol: El sistema debe validar las credenciales contra la base de datos y redirigir correctamente a las rutas protegidas: /dashboard/admin, /dashboard/tecnico o /dashboard/empleado según el campo rol definido para el usuario.

Integridad de Datos en el Flujo de Incidencias: La base de datos debe reflejar con precisión las relaciones entre usuarios, categorías y tickets (incidencias), respetando las claves foráneas y restricciones de integridad definidas en el esquema SQL.

Gestión Completa del Ciclo de Vida del Ticket: El flujo operativo debe permitir que un empleado cree una incidencia (estado 'abierta'), un técnico la procese ('en proceso') y finalmente se cierre al ser solucionada ('resuelta').

Gestión de Documentación Técnica (PDF): El sistema permitirá adjuntar archivos PDF a las incidencias utilizando Supabase Storage. Esta herramienta se encargará de alojar los archivos físicos, mientras que en la base de datos solo se almacenará la URL de referencia en la columna url_adjunto. El uso de esta herramienta está sujeto a límites de almacenamiento (máximo 1 GB total) y de tamaño por archivo individual (50 MB), garantizando así que la base de datos no se sobrecargue con datos binarios.

Interfaz Adaptable y Reactiva: La interfaz debe ser totalmente funcional tanto en modo claro como oscuro, manteniendo la coherencia visual mediante variables de CSS y proporcionando feedback inmediato al usuario a través de mensajes de error, estados de carga y confirmaciones visuales de conexión.

Sistema de Trazabilidad y Comentarios: La funcionalidad de comentarios debe permitir una comunicación fluida y persistente entre empleados y técnicos, vinculando cada mensaje a su ticket correspondiente y autor a través de la tabla de comentarios.

Rendimiento y Estabilidad de Conexión: El sistema debe mantener una comunicación constante con la base de datos en la nube mediante el uso del Session Pooler. Esto garantiza tiempos de respuesta rápidos y evita errores de conexión derivados de configuraciones de red locales restrictivas (como el uso de IPv4).

3. Información para ejecutar el proyecto

Para que todo funcione correctamente después de descargar el proyecto, hay que seguir los siguientes pasos:

Enlace del proyecto →https://github.com/sursocarlos/CentralTicket

1. Requisitos previos
Lo único que hace falta es tener instalado **Node.js** (yo he usado la **versión 18**, así que recomiendo esa o una más nueva).

2. Instalación de librerías
Como las carpetas de dependencias (**node_modules**) no se suben al repositorio para que no pese tanto, hay que instalarlas a mano en las dos partes del proyecto:

Entra en la carpeta **/backend** desde la terminal y lanza un: **npm install.**

Haz lo mismo en la carpeta **/frontend:** entra y ejecuta también: **npm install.**

3. Configuración de la base de datos
Se ha incluido un archivo .env.example en la raíz del backend. El evaluador debe renombrarlo a .env y sustituir los valores de ejemplo por sus propias credenciales de Supabase para habilitar la conexión.

4. Cómo lanzar la aplicación
Para que el sistema funcione completo, hay que tener dos terminales abiertas a la vez:

Terminal 1 (Servidor): Entra en **/backend** y escribe node **index.js**. Verás un mensaje confirmando que el servidor está escuchando.

Terminal 2 (Interfaz): Entra en **/frontend** y escribe **npm run dev.** Esto te dará el enlace para abrir la web en el navegador.

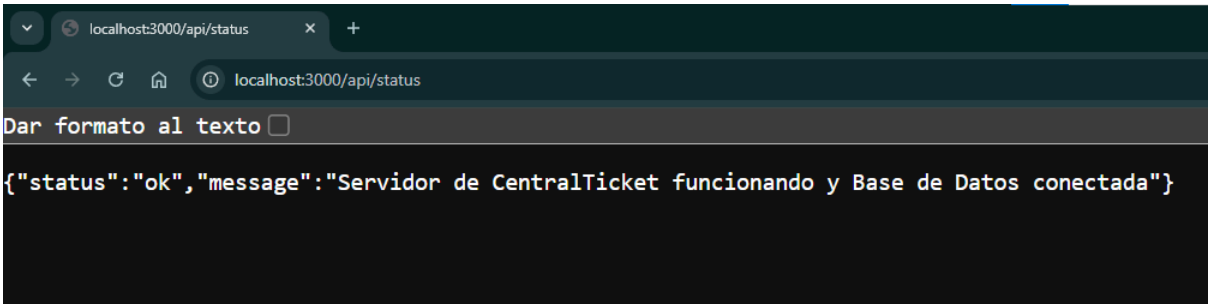
5. Cuentas de prueba
He dejado creados tres usuarios en la base de datos para poder probar los diferentes roles directamente (puedes ver el script de creación en la carpeta **database/schema.sql)**:

- **Perfil Admin:** admin@centralticket.com / 1234
- **Perfil Técnico:** tecnico@centralticket.com / 1234
- **Perfil Empleado:** empleado@centralticket.com / 1234

4. Verificación del prototipo

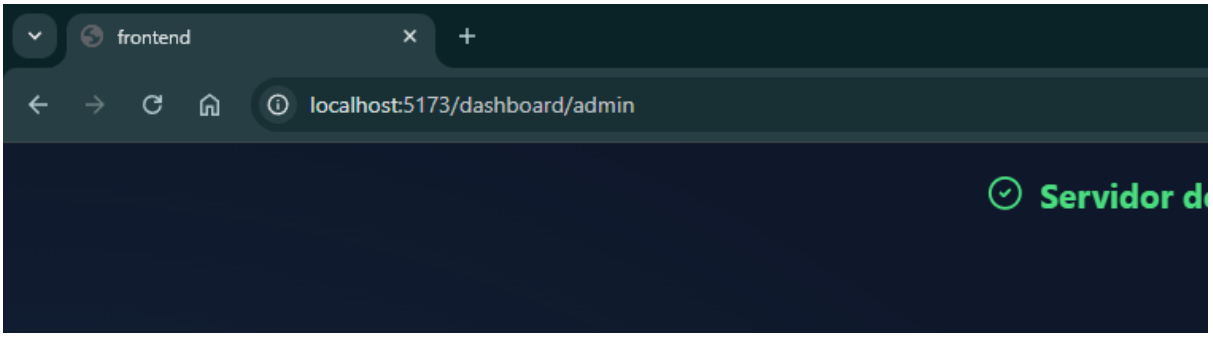
- Check 1: Comprobación de la API (Heartbeat):
Qué hacer: Con el servidor encendido, abrir el navegador en http://localhost:3000/api/status.

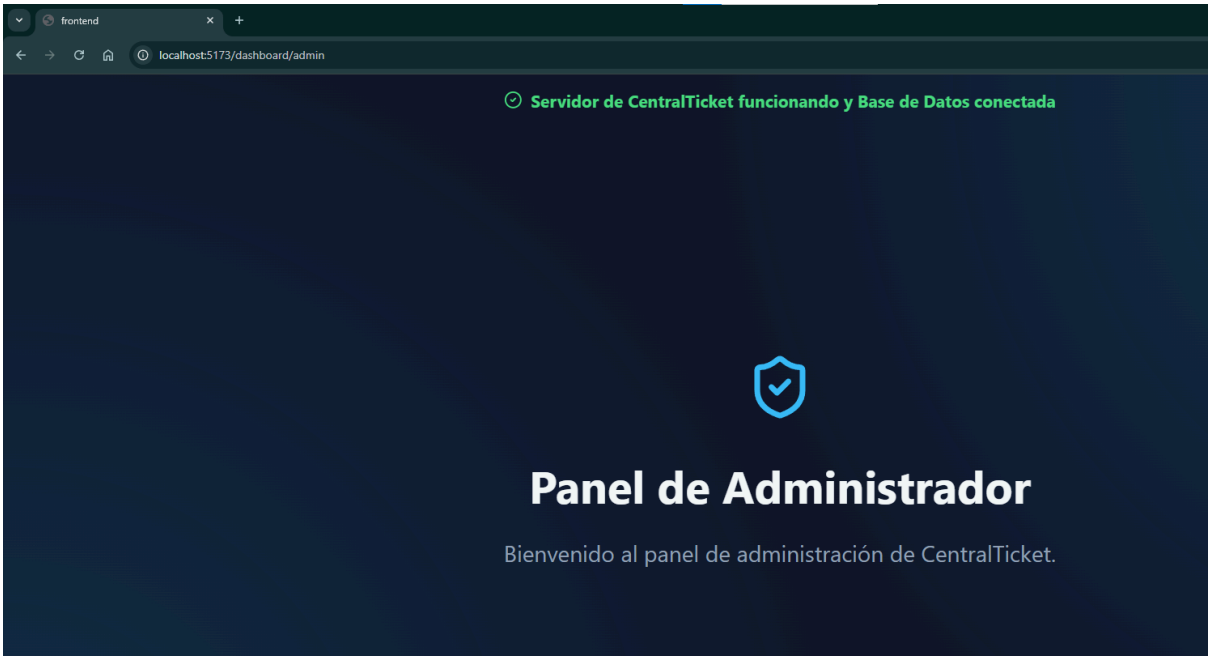
Qué debe pasar: Debe aparecer un mensaje JSON que diga {"status": "ok", "db": "connected"}. Esto demuestra que el servidor vive y llega a la base de datos.



- Check 2: Lógica de roles (Redirección)
Qué hacer: Tras el login exitoso, verificar que la URL cambia (ej: de /login a /dashboard/admin).

Qué debe pasar: Esto confirma que el frontend sabe leer el campo rol de la base de datos y actúa en consecuencia.



- Check 3: Validación de credenciales reales
Qué hacer: En el formulario de Login, meter el usuario admin@centralticket.com y la clave 1234.

Qué debe pasar: El sistema no debe dar error de "Usuario no encontrado", confirmando que la consulta a la tabla usuarios funciona.